

Project Report Anti Timpa

Prepared by:

2026 October

Chapter 1: Introduction & Background

1.1 Scale of the National QRIS Ecosystem and Physical Exposure

Since its nation-wide implementation by Bank Indonesia (BI) and ASPI, QRIS has transformed Indonesia's digital financial inclusion. By mid-2026, the ecosystem encompasses:

Total Registered Merchants: >47,000,000 merchants across 38 provinces.

MSME Share: $\approx 96.7\%$, dominated by informal stalls (warung), traditional markets, and small food vendors.

Deployment Form Factor: Over 95% of MSME payment touchpoints utilize static paper-printed, acrylic-encased, or vinyl adhesive QR decals affixed to counters or walls.

While dynamic QRIS displays (integrated into Point-of-Sale terminals or digital screens) provide session-specific amount encoding, static QRIS decals represent the foundational backbone of daily retail commerce in Indonesia. However, physical accessibility creates an uncontrolled attack surface.

1.2 The "QRIS Tempelan" Attack Vector

The primary physical-layer attack vector target of Anti Timpa is the adhesive QR overlay exploit (QRIS Tempelan). An attacker prints a batch of self-adhesive vinyl QR decals containing their own valid merchant payload (or a mule account registered under a fraudulent identity). The attacker physically visits victim merchants during peak operating hours and pastes the sticker directly over the genuine QR decal.

Because the sticker completely obscures the underlying QR code, standard optical decoders (such as ZXing) decode only the attacker's payload. The victim's mobile banking app reads the payload, presents the attacker's merchant name, and executes the transfer upon user PIN authorization. The legitimate merchant receives no notification, and the transaction fraud is discovered only during end-of-day ledger reconciliation.

1.3 Why Traditional Security Controls Fail at the Point-of-Scan

Conventional cyber-defense paradigms fail entirely when confronted with physical overlay tampering:

Server-Side Firewalls & Payment Gateways: Cloud payment gateways process incoming API requests containing valid digital signatures, authentic account IDs, and intact CRC-16 checksums. The server cannot determine whether the user scanned a QR code belonging to the physical store or an adhesive sticker pasted over it.

Transport Layer Encryption (TLS/HTTPS): TLS secures the communication tunnel between the user's smartphone and the bank's servers. It provides zero visibility into the physical optical medium captured by the camera lens.

Database Pre-Registration & Image Hashing: Cloud lookup systems that attempt to compare scanned QR images against a central registry of merchant photographs require massive cloud database latency, high network bandwidth, continuous cellular connectivity, and suffer severe false-positive rates due to ambient lighting variations, wear-and-tear, and angle tilt.

Anti Timpa addresses this fundamental gap by introducing On-Device Physical Artifact Verification, embedding optical physical edge detection and structural payload validation directly inside the local scanning pipeline.

Chapter 2: Solution Overview & Market Differentiation (USP Analysis)

2.1 Solution Overview: Anti Timpa

Anti Timpa is an edge-native computer vision and payload validation engine built in pure Rust (anti_timpa_lib) and delivered across platforms via Tauri v2. It targets zero-trust operational

environments and runs completely offline, analyzing live camera frames and payload streams on-device. No image data or position coordinates are transmitted to any server.

2.2 Three-Layer Defense Architecture

Layer 1: Optical Tamper Analysis

Layer 1 analyzes the physical camera crop surrounding the QR code bounding box (qr_bbox). Because a sticker overlay is physically applied by hand onto an existing card or sheet, its physical boundary introduces sharp edge discontinuities into the mandatory EMVCo/ISO 18004 Quiet Zone (the blank paper margin surrounding the QR symbol).

Layer 1 computes:

Spatial Edge Density (p_e): Evaluated over a calibrated rectangular quiet-zone ring (QUIET_ZONE_SEARCH_FRACTION = 0.12).

Temporal Glare Variance (σ_{glare}^2): Analyzes multi-frame specular highlights across thermal sticker vinyl versus base paper.

Texture Discontinuity (σ_{tex}^2): Measures local intensity variance across strided 3x3 sub-windows.

Layer 2: EMVCo MPM Structural & Semantic Validation

Layer 2 parses the raw QRIS string into standard EMVCo Tag-Length-Value (TLV) trees. It enforces strict structural and semantic consistency rules:

Checksum Verification: Evaluates CRC-16/CCITT-FALSE (polynomial 0x1021, seed 0xFFFF) over Tag 00 through Tag 6304.

Contextual QR Check: Flagging dynamic QR codes (Tag 01 = "12") when captured via physical camera scans (where static Tag 01 = "11" is mandated).

Charity MCC Misrepresentation: Detects fraudulent registration where non-profit/charity MCCs (e.g., 8661 Religious Organizations, 8398 Charitable Services) are paired with commercial merchant keywords (warung, toko, store, cell).

Layer 3: Geographic Plausibility Engine

Layer 3 cross-references the merchant's declared city in EMVCo Tag 60 against coarse device location. It resolves city strings into canonical gazetteer coordinates, applies administrative prefix stripping (KOTA, KABUPATEN), and calculates great-circle distance via the Haversine formula. Rather than triggering binary false positives for commuting users, Layer 3 employs a Tiered Risk Model (DISTANT_KM = 150.0 km) capping location mismatches below hard veto thresholds unless corroborated by Layer 1 or Layer 2 anomalies.

Chapter 3: Proof of Concept (PoC) Implementation & Repository References

3.1 Stack and why

Component	Technology	Why this choice
Core analysis	Rust	Runs unchanged on desktop/Android/iOS; no GC pauses during frame analysis; memory-safe while parsing attacker-controlled input
App shell	Tauri 2	Native app with a strict CSP and no bundled Node runtime; the security boundary is the OS
UI	React 18 + TypeScript	Fast iteration; typed IPC contract
QR decode	rqrr (pure Rust)	No external vision library; decodes in-process

Component	Technology	Why this choice
Image import	image crate	Runs the optical layer on a photo without a camera
QR encode	qrcode (dev only)	Synthetic backend emits real, decodable symbols
Camera (desktop)	nokhwa	V4L2 / AVFoundation / MSMF
Camera (mobile)	CameraX (JNI) / AVFoundation (C-ABI)	Frames pushed from native into a Rust slot

3.2 Layer 1 — Optical tamper analysis (the core contribution)

Implemented in `src-tauri/src/layer1_optical.rs`.

Signal 1 — edge density in the detection ring. A QR symbol requires a blank margin. An attacker's sticker must be larger than the symbol, otherwise the original code would still decode around it. The sticker's border therefore lands in the ring just outside the symbol, as a straight high-contrast line where blank paper should be. Measured as the fraction of pixels in that ring whose Sobel gradient exceeds a noise floor.

The ring width is the most sensitive parameter, fitted by measurement rather than chosen. The recorded sweep (`EDGE_MAGNITUDE_THRESHOLD = 24`):

Ring band (× symbol side)	Clean QR	QR with sticker
0.03	0.1394	0.1394
0.05	0.0797	0.5079
0.08	0.0490	0.4219
0.12 (selected)	0.0321	0.2767
0.14	0.0932	0.2969

At 0.03 and 0.14 the ring clips the paper-to-background boundary of the sheet itself, so a clean code reads as heavily anomalous. 0.12 sits between both failure modes and retains $\approx 8.6\times$ separation.

Signal 2 — specular glare in the ring. A sticker is a different material (glossy thermal print, adhesive film). Across a burst of frames the share of specular pixels in the ring varies, where paper's does not.

Signal 3 — texture discontinuity (measured and reported, given no weight). The peak 3×3 local variance over the symbol interior saturates on any readable QR ($9800 = 99^2$) and cannot discriminate, so it is returned in `Layer1Result` for future re-fitting but does not affect the score.

Scoring. $l1_score = 0.70 \times \text{spatial} + 0.30 \times \text{disagreement}$, where $\text{spatial} = 0.75 \times \text{edge} + 0.25 \times \text{glare}$, plus a bounded temporal bonus (`TEMPORAL_BOOST = 0.35`) that can raise a score but never convict alone. Measured on fixtures: clean 0.000 / LOW RISK, sticker 0.371 / CAUTION.

3.3 Layer 1 calibration limits

The sticker fixture does not reach HIGH RISK, and the thresholds were not lowered to make it. The fixture's overlay covers part of the symbol, a moderate violation; forcing HIGH RISK would require thresholds that start flagging clean codes. Because a missed overlay costs real money while a false alarm costs a re-scan, thresholds lean sensitive — but not so far that clean codes trip. The test asserts "at least CAUTION", and the reasoning is recorded on the constant.

3.4 Layer 2 — EMVCo payload integrity

Implemented in `src-tauri/src/layer2_emvco.rs`. Flat and nested TLV parsing (tags 26–51), CRC-16/CCITT-FALSE, and four risk rules: format indicator; currency and country; dynamic QR in a physical-scan context; and charity-MCC paired with a commercial merchant name. A CRC

failure is a hard veto forcing the combined score to 1.0 — the one case where a payload check is genuinely conclusive.

3.5 Layer 3 — Geographic plausibility (rebuilt)

Implemented in `src-tauri/src/layer3_geofence.rs`, with `src-tauri/src/geo_table.rs` for offline city resolution.

The problem with the original rule. It was `client_city != merchant_city → score 1.0 → HIGH RISK`. Because the combined score is $\max(I1, I2, I3)$, that alone forced the headline verdict to HIGH RISK, firing hardest on travellers scanning a legitimate national-chain code. Worse, it did not address the attack: an overlay copies Tag 60 verbatim, so the merchant city is genuine and Layer 3 sees nothing.

The rebuilt layer replaces the flat rule with classification:

Outcome	Score	Meaning
MATCH	0.00	Names agree after normalisation
SAME_METRO	0.00	Different name, same place (KOTA/KABUPATEN levels, aliases)
NOT_COMPARABLE	0.00	Tag 60 is non-ASCII, or a non-geographic marker (ONLINE)
NOT_EVALUATED	0.00	An input is missing — explicitly not a pass
DIFFERENT_CITY_NEARBY	0.40	Far apart by name, within ~150 km, or a land-border city
DIFFERENT_CITY_DISTANT	0.65	Different name and measurably far
DIFFERENT_CITY_UNBOUNDED	0.55	Different name, no reference data to bound it

Distance is Haversine, computed from a device fix when the user opts in, otherwise from city reference points. (0,0) and out-of-range fixes are rejected as placeholders. No path in this layer can reach the HIGH RISK band — enforced by a test across four mismatch shapes so a future retune cannot silently reintroduce the veto.

A second, independent bug. Layer 3 reported NOT RUN on every scan for two unrelated reasons: (1) the UI called `navigator.geolocation`, which a Tauri WebView on Linux cannot back with a real service, so the call errored and the scan got no location while the UI still showed a location checkbox; and (2) a typed city resolved no coordinates, because the payload-only path performed no lookup. The first was fixed with `@tauri-apps/plugin-geolocation` on mobile; the second by resolving the typed name against the bundled offline table in `geo_table.rs`.

Platform	Location source	Why
Android / iOS	OS service via geolocation plugin	The only true position, with a real permission prompt
Desktop	Offline city table, from the typed city	Desktop has no OS location service. IP geolocation was rejected — it would send the user's position to a third party
Browser (npm run dev)	<code>navigator.geolocation</code> , best-effort	Keeps UI work possible outside Tauri; not a supported path

3.6 Coverage reporting and named findings

Implemented in `src-tauri/src/lib.rs` (`build_findings` and `coverage_summary`) and surfaced by `src/components/CoverageBanner.tsx` and `src/components/FindingsList.tsx`. `ScanSnapshot` carries two fields that exist to stop a partial scan being misread as clean: `coverage` (which layers ran, and `complete: bool`) and `findings` (a ranked list of rules that fired, each with a stable code such as `L2_CRC_MISMATCH` or `L1_OPTICAL_TAMPER`). The `CoverageBanner`

component renders a partial scan as a warning, not a success, and names exactly what was skipped, because a payload-only scan of an overlaid QR legitimately returns LOW RISK while the only check that could have caught the attack never ran.

User interface: the app (`src/App.tsx` and `src/components/`) has two modes. Simple mode shows the verdict, merchant name and city, risk value and plain-language findings. Technical mode adds per-layer scores, the raw payload, optical metrics and camera diagnostics. A scan that ran only the payload layers is shown with a warning banner rather than a green verdict, so a partial check is never presented as safe.

3.7 Camera pipeline

Implemented in `src-tauri/src/lib.rs` (`capture_and_analyze`) on top of the camera backends in `src-tauri/src/camera/` (`desktop.rs`, `synthetic.rs`, `mobile/android.rs`, `mobile/ios.rs`) with the platform bridges in `android/CameraBridge.kt`, `android/CameraFrameAnalyzer.kt` and `ios/CameraBridge.swift`. `capture_and_analyze` captures a burst of 4 frames (`DEFAULT_BURST_FRAMES`), because a single frame cannot distinguish a moving highlight from a static bright patch. Burst capture is best-effort: if the device delivers fewer frames, Layer 1 degrades to spatial-only and reports that the temporal term was skipped, rather than failing the scan. Layer 1 runs only when a QR was decoded, because without a bounding box there is no margin to measure, and reporting a score anyway would be the false-clean result the layer exists to prevent.

3.8 Image import — a validation path, not a product feature

Implemented in `src-tauri/src/image_import.rs`. This module is the way Layer 1 gets validated against real photographs and demonstrated on a machine with no camera; it is not the primary detection path. A single imported photo cannot support the temporal glare check, which needs a burst of the same untouched scene. The obvious cheat — re-analysing one image four times and calling the zero variance "no glare risk" — is refused: `ImportInfo.imported_frame_has_burst` is hard-coded false, Layer 1 runs spatial-only, and the skipped temporal check appears in the warnings.

Format-agnostic decoding via `ImageReader::with_guessed_format()`, because a file picked from disk often has no useful extension.

A resolution band (`MAX_IMPORT_DIMENSION = 1600`): a modern phone photo is 4000+ px and ~48 MB decoded; Layer 1's geometry is scale-invariant, so downscaling costs no signal and keeps imported frames in the same regime as camera frames.

Conditional resampling — upscaling and unnecessary resize are avoided, since a needless resample would blur the fine margin detail Layer 1 exists to measure.

3.9 Evidence layer

Implemented in `src-tauri/src/history.rs`. A session-scoped scan log with a tamper-evident hash chain: each entry's hash covers its own canonical encoding plus the previous entry's hash, so editing or removing an entry breaks every hash after it. Field values are length-prefixed to close boundary-ambiguity collisions, and the buffer is capped at 200 entries.

3.10 Verified status

Check	Command	Result
Core tests	<code>cargo test --no-default-features --features "jni-bridge,qr-encode" --lib</code>	153 passed, 0 failed
Core tests (no encoder)	<code>cargo test --no-default-features --features jni-bridge --lib</code>	141 passed, 0 failed
Desktop compile	<code>cargo check --features desktop-camera,jni-bridge,geolocation</code>	Clean, no warnings
Frontend types	<code>npx tsc --noEmit</code>	Clean
Frontend build	<code>npm run build</code>	Clean

Check	Command	Result
Core tests (iOS bridge)	cargo test --no-default-features --features "jni-bridge,ios-bridge,qr-encode" --lib	162 passed, 0 failed

The qr-encode feature is off by default, so the shipping binary never links an encoder: the product only ever reads QR codes. Fixture results in the current build (real symbol rendered by the synthetic backend, 640×480):

```
clean QR      : payload=106B  edge=0.0162  score=0.000  band=LOW RISK
sticker QR    : payload=106B (IDENTICAL) edge=0.2767  score=0.371  band=CAUTION
CRC tampered  :                                           score=1.000  band=HIGH RISK (veto)
traveller     : L3 DIFFERENT_CITY_DISTANT                score=0.650  band=CAUTION (no veto)
imported photo: downscaled 3200px -> 1600px, decodes, L1 runs spatial-only
```

Chapter 4: Technical Architecture & Feasibility

4.1 System architecture

The full data path is shown in Figure 4. The device performs every step; no component talks to a network.

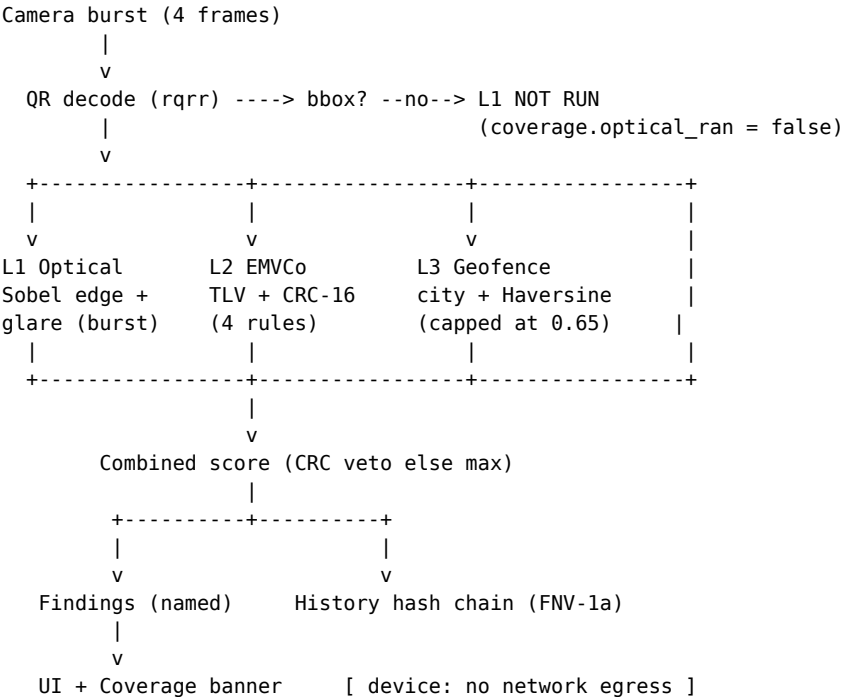


Figure 4. System architecture and data flow

4.2 Data flow and trust boundaries

The critical architectural property is that no trust boundary is crossed at runtime. The only boundary is the IPC seam between the React WebView and the Rust core, which is an in-process channel in the same application:

Boundary	Protocol	Data crossing	Trust assumption
WebView → Rust core	Tauri IPC (in-process)	Payload string, frame metadata, snapshot JSON	Same process, same user
Rust → camera	OS device API	Pixel buffers	OS-enforced permissions
Rust → network	none	nothing	CSP default-src 'self'

Raw pixels never cross into the WebView for analysis. The preview path sends downscaled JPEG data URLs purely for display, and analysis always runs on the full-resolution frame in Rust — analysing a downscaled JPEG would destroy the fine edge detail Layer 1 depends on.

4.3 Why Rust

Requirement	Consequence of the choice
One analysis implementation for desktop + Android + iOS	No platform-specific reimplementations to drift out of sync
Bounded latency during frame analysis	No GC pause in the middle of a burst
Parses attacker-controlled input (TLV from a QR)	Memory safety removes the injection class entirely
Camera frames are large; per-frame copies are costly	Pixels are read from the native direct buffer without a JVM-heap copy; one ownership copy remains so the frame outlives the camera callback

4.4 The synthetic backend as a feasibility strategy

camera/synthetic.rs generates deterministic frames containing a QR symbol with configurable sticker anomaly and glare. The backend makes the optical layer testable: every Layer 1 threshold was fitted against measurable fixture output, and a test fails if a constant is retuned so the limit no longer separates the clean and sticker fixtures. It renders real QR codes via the qrcode crate behind an off-by-default qr-encode feature, so the shipping binary never links an encoder it does not use. Layer 1 is therefore tested on frames that actually contain a QR, and the fixture carries a real QRIS payload, so the identical-payload property of the overlay attack is an executable assertion rather than a claim.

4.5 Threshold selection and the cost asymmetry

Error	Cost	Mitigation
False negative (missed overlay)	Customer pays the attacker. Direct financial loss.	Thresholds lean sensitive; HIGH RISK band kept reachable
False positive (clean code flagged)	Re-scan, mild annoyance, erosion of trust	Calibrated limits ~2× the clean measurement; HIGH RISK reserved for severe violations

The false negative is far more expensive, so the design accepts a CAUTION verdict on moderate violations rather than demanding a certainty the sensor cannot give.

4.6 Known limitations

The city table is small (~12 entries covering the demo corridor), not the ~514 Indonesian kabupaten/kota.

Layer 1 cannot see through blur or at extreme angles. quiet_zone_truncated reports when the margin was not measurable, but does not fix it.

4.7 Feasibility summary

Every external dependency is a mainstream, maintained crate or plugin. The architecture requires no cloud service, no account system, no proprietary SDK, and no hardware beyond a camera the device already has. The hardest remaining work is data collection and threshold fitting, not engineering risk.

Chapter 5: Security Architecture & Intellectual Property Potential

5.1 STRIDE Threat Modeling Matrix

A formal threat model was constructed following the STRIDE methodology to evaluate physical and digital attack vectors against Anti Timpa in physical retail environments.

STRIDE Threat Category	Threat Scenario	Physical / Digital Vulnerability	Anti Timpa Defensive Implementation
Spoofing	Attacker impersonates legitimate merchant via overlay decal.	Attacker pastes valid static QRIS of mule account over victim merchant QR.	Layer 1 Quiet-Zone Sobel Edge Density & Layer 3 Geofence City Tier Matching.

STRIDE Threat Category	Threat Scenario	Physical / Digital Vulnerability	Anti Timpa Defensive Implementation
Tampering	Attacker modifies payload bytes (e.g., changes recipient account ID in place).	Man-in-the-middle payload edit or manual byte manipulation.	Layer 2 CRC16/CCITTFALSE Checksum Hard Veto (combined_score = 1.0).
Repudiation	Fraudulent merchant denies tampering or user claims false detection.	Absence of historical scan audit trail.	Tamper-Evident Hash Chain (history.rs) locking sequence, preview, and scores.
Information Disclosure	Leakage of sensitive merchant transactions or user location tracking.	Unauthorized cloud telemetry transmission or location harvesting.	100% On-Device Processing Guarantee; zero network egress of camera frames/GPS.
Denial of Service	Malformed QR payload causes decoder loop or app crash.	Nested recursive TLV exploit or memory exhaust payload.	Bounded TLV Depth & Rust memory-safe slice parsing with strict length guards.
Elevation of Privilege	Native buffer overflow via JNI camera buffer passing.	Passing invalid memory pointers from Java to C/Rust native layer.	GetDirectBufferAddress validation and checked slice boundaries in camera/mobile/android.rs and camera/mobile/ios.rs

5.2 Security Guarantees & Privacy Protection

Anti Timpa enforces absolute privacy by design to comply with Indonesian Personal Data Protection (UU PDP No. 27/2022) mandates:

Zero-Egress Local Processing Boundary

Anti Timpa enforces a strict zero-egress architectural boundary. High-resolution camera pixel buffers (Android CameraX and iOS AVFoundation), intermediate grayscale transformations, raw EMVCo payload strings and coarse GPS coordinates remain within local process memory. The Rust runtime performs all Sobel convolution, recursive Tag-Length-Value parsing and Haversine distance calculation at the edge, reading pixel buffers directly via GetDirectBufferAddress with a single ownership copy kept for memory safety. Because the core detection pipeline opens no network socket, the application exfiltrates no purchasing habits, location traces or merchant imagery, consistent with Indonesian Personal Data Protection Law (UU PDP No. 27/2022).

5.3 Intellectual Property Potential

No patent search was performed, so this is reasoning, not a legal opinion. The individual techniques (Sobel edge density, specular-glare variation across frames) are long-established computer vision and are not novel on their own, so a patent on the detection method is unlikely; the project relies on execution rather than exclusivity. The defensible asset is the labelled field-capture dataset and the thresholds fitted to it, which are hard to reproduce and are published only as aggregate results. Publishing the method in this report is a deliberate defensive-disclosure choice: it costs little where patentability is weak, and it establishes prior art.

Chapter 6: Scalability & Deployment Readiness

6.1 Hardware & Operating System Requirements

Anti Timpa is engineered for maximal backward compatibility across low-cost mobile devices deployed across Indonesia.

Anti Timpa targets the low-cost Android devices that dominate Indonesian retail. The practical minimums are: Android 8.0 (API 26) or newer, which is the CameraX baseline, or iOS 14 or newer for the AVFoundation bridge; desktop builds run on Linux, macOS and Windows. The analysis needs only a rear camera that can deliver RGBA8888 frames, a short burst of four

frames at a bounded resolution (1280x720), and no dedicated accelerator. No model is bundled and no cloud dependency exists, so the install stays small (roughly 10-15 MB) and the only persistent local file is the JSON scan history. No connectivity is required at scan time; the app works in airplane mode.

6.2 Mobile SDK Integration (C-ABI)

The core crate (`anti_timpa_lib`) exposes a C-ABI in `src-tauri/src/sdk.rs` with three functions: `antitimpa_version`, `antitimpa_analyze_payload` (JSON in, JSON out) and `antitimpa_free_string`. A bank or PSP links the native library and calls one function from its existing scan screen, with no change to the core banking stack. The standalone app and any embedded copy call the same analyzer (`src-tauri/src/lib.rs`, `analyze_payload_snapshot`), so both always return the same verdict. The Rust library builds as a static or dynamic library; scripted packaging for `.aar` (Android) and `.framework` (iOS) is provided in `sdk/`, but has not been built in this environment because it requires the Android NDK and Xcode respectively. The C-ABI functions themselves are implemented and covered by unit tests.

6.3 Field Validation and Calibration Roadmap

All current Layer 1 thresholds are fitted to synthetic fixtures. The next milestone is field calibration: (1) collect 1,000 or more photographs of real QRIS decals from MSMEs under bright, dim, folded and dirty conditions, alongside a range of overlay stickers (HVS paper, thermal print and glossy vinyl); (2) re-fit the four optical constants using the image-import path (`analyze_image_bytes`) so the clean/overlay separation holds on real captures, where the raw metrics are already reported in `Layer1Result` and only the constants change; and (3) extend the offline city table toward the full set of Indonesian kabupaten/kota. A single session with a printer and a few phones closes the largest remaining unknown.

6.4 Conclusion

Anti Timpa addresses one thing existing controls ignore: the physical object. It runs entirely on the customer device, holds no account and contacts no server, so its privacy and security properties are structural rather than policy promises. The engineering risk is low and the remaining work is data collection and calibration. Its coverage reporting states what was not checked, which is the minimum for a tool people rely on before sending money.